Smart Outcome Predictor - Theory Concepts

Subject: Supervised Learning - Ensemble Methods
Project: Smart Outcome Predictor

1. Ensemble Learning Methods

> **Definition:** Ensemble learning is a machine learning technique where multiple models (called base learners or weak learners) are combined to produce a single, stronger predictive model. The key idea is that by aggregating predictions from several models, we can achieve better generalization and more stable results than any single model alone.

Why are they effective?

- **Variance Reduction:** Methods like bagging average out the noise from individual models, reducing overfitting.
- **Bias Reduction:** Methods like boosting iteratively correct errors, reducing underfitting.
- **Improved Stability:** A single model's performance can fluctuate across different data samples. An ensemble smooths out these fluctuations.
- **Better Generalization:** Combining diverse models captures different aspects of the data, leading to better performance on unseen data.

2. Bagging (Bootstrap Aggregating)

> **Definition:** Bagging is an ensemble technique that creates multiple bootstrap samples (random subsets with replacement) from the training data, trains an independent base model on each sample, and then aggregates their predictions through voting (classification) or averaging (regression).

How it works:

1. Draw N random samples (with replacement) from the training data — each sample is called a bootstrap sample.
2. Train one base model (typically a decision tree) on each bootstrap sample.
3. For classification: take a majority vote of all models. For regression: take the average prediction.

```
Bagging prediction = (1/N) * Σ f_i(x), where f_i is the i-th base model
```

Key Property: All base models are trained independently (in parallel). Bagging primarily reduces variance.

Example: Random Forest is the most famous bagging algorithm — it bags decision trees with additional feature randomization.

3. Boosting

> **Definition:** Boosting is a sequential ensemble technique where each new model is trained to correct the errors made by the previous models. Models are added one at a time, and each subsequent model focuses more on the hard-to-predict samples.

Key Property: Models are trained sequentially (each one depends on the previous). Boosting primarily reduces bias.

3.1 AdaBoost (Adaptive Boosting)

- Uses decision stumps (depth-1 trees) as weak learners by default.
- After each round, misclassified samples receive higher weights so the next learner focuses on them.
- Each learner gets a weight (alpha) based on its accuracy — better learners get more influence.
- **Weakness:** Sensitive to noisy data and outliers because it keeps increasing their weights.

3.2 Gradient Boosting

- Instead of reweighting samples, it fits new trees to the **residual errors** (negative gradient of the loss function).

- Uses a learning rate parameter to control how much each new tree contributes.
- More flexible than AdaBoost — supports different loss functions (MSE, MAE, log-loss, etc.).
- **Weakness:** Slower to train because trees are built sequentially (cannot be parallelized).

3.3 LightGBM (Light Gradient Boosting Machine)

- Developed by Microsoft. Uses **histogram-based splitting** for faster training.
- Grows trees **leaf-wise** (best-first) instead of level-wise, which leads to deeper, more accurate trees.
- Implements GOSS (Gradient-based One-Side Sampling) to focus on samples with larger gradients.
- Implements EFB (Exclusive Feature Bundling) to reduce feature dimensionality.
- **Key advantage:** Much faster training time, especially on large datasets.

3.4 XGBoost (Extreme Gradient Boosting)

- Developed by Tianqi Chen. Adds **L1 (Lasso) and L2 (Ridge) regularization** to the objective function.
- Uses a second-order Taylor expansion of the loss function for more precise optimization.
- Handles missing values natively during tree construction.
- Supports parallel tree construction and distributed computing.
- **Key advantage:** Built-in regularization prevents overfitting better than standard Gradient Boosting.

4. Voting

> **Definition:** Voting is an ensemble technique that combines predictions from multiple different types of models trained on the same dataset.

Hard Voting

- Each base model casts a vote for a class.
- The final prediction is the class with the most votes (majority voting).
- Simple but effective when base models are diverse.

Soft Voting

- Each base model outputs class probabilities.
- The probabilities are averaged across all models.
- The class with the highest average probability is chosen.
- Generally better than hard voting because it uses more information (probabilities vs. just class labels).

```
Hard Voting: y = mode(y_1, y_2, ..., y_n)
Soft Voting: y = argmax(avg(P_1(y), P_2(y), ..., P_n(y)))
```

5. Stacking (Stacked Generalization)

> **Definition:** Stacking is a two-level ensemble method. Level 0 consists of diverse base models whose predictions become features for a Level 1 meta-learner that learns the optimal way to combine them.

How it works:

1. Train several diverse base models (Level 0) on the training data.
2. Use cross-validation to generate out-of-fold predictions from each base model.
3. These predictions become the input features for a meta-learner (Level 1).
4. The meta-learner (e.g., Logistic Regression, Ridge) learns how to best combine the base model outputs.

Key advantage: The meta-learner can learn which base model to trust more in different situations, making it more powerful than simple voting.

6. Bias-Variance Trade-off

> **Definition:** The bias-variance trade-off describes the tension between a model's ability to fit training data (low bias) and its ability to generalize to new data (low variance).

- **High bias** = model is too simple, underfits (e.g., linear model on non-linear data).
- **High variance** = model is too complex, overfits (e.g., deep decision tree memorizing training data).

```
Total Error = Bias² + Variance + Irreducible Noise
```

How ensembles address it:

Method	Reduces	How
Bagging	Variance	Averages predictions from multiple high-variance models
Boosting	Bias	Sequentially corrects errors from high-bias models
Stacking	Both	Meta-learner optimizes the combination of diverse models

7. Key Evaluation Metrics

Classification Metrics

Metric	Formula	What it measures
Accuracy	(TP + TN) / Total	Overall correctness
Precision	TP / (TP + FP)	Of predicted positives, how many are correct
Recall	TP / (TP + FN)	Of actual positives, how many were found
F1-Score	2 * (Precision * Recall) / (Precision + Recall)	Harmonic mean of precision and recall
ROC-AUC	Area under ROC curve	Model's ability to distinguish between classes

Regression Metrics

Metric	Formula	What it measures

MAE	$(1/n) * \Sigma\|y\_actual - y\_predicted\|$	Average absolute error
RMSE	$\sqrt{(1/n) * \Sigma(y\_actual - y\_predicted)^2}$	Root mean squared error (penalizes large errors)
R^2 Score	$1 - (SS\_res / SS\_tot)$	Proportion of variance explained by the model (1.0 = perfect)